

Risk-Sensitive Distributional Actor-Critic for Sales Dialogue Control: A Mathematically Grounded Benchmark Against Frontier Prompted Controllers

Abhiram Venkat Sai Adabala, Shikhar Veeramachineni, Ashok Jupally
Independent Research

Abstract—Sales conversations are sequential control problems with delayed rewards, partial observability, asymmetric downside risk, and non-stationary customer behavior. Prompted frontier language models can generate high-quality responses, but they are not trained to optimize a dialogue policy directly for conversion, calibration, or tail-risk control. We formulate sales dialogue optimization as a partially observed Markov decision process and develop a risk-sensitive distributional actor-critic that learns a quantile approximation to the return distribution rather than a scalar value function. The resulting policy update depends on both the mean return and the lower tail of the return distribution, allowing explicit control over catastrophic outcomes such as premature closes, objection escalation, and fatigue-driven conversation collapse. We also present a controlled comparison framework spanning GPT-5, GPT-5.1, Claude Sonnet 4.5, and the proposed Sales RL Agent. To make the framework reproducible, we provide A100-ready training and inference scripts together with a synthetic dialogue simulator. In a three-seed benchmark on the released simulator, the distributional controller reduces episode-return volatility by 9.3% and improves CVaR_{0.1} from -1.465 to -1.140 , while the scalar A2C baseline attains higher mean reward. The empirical picture is therefore not that distributional RL always increases average reward, but that it yields a more conservative and substantially better downside-risk profile, which is the more relevant operating point for safety-critical revenue workflows.

Index Terms—Distributional reinforcement learning, actor-critic, sales agents, dialogue control, risk-sensitive optimization, LLM agents.

I. INTRODUCTION

Autonomous sales agents must reason over a sequence of conversational turns rather than a single static classification input. A good controller decides when to ask discovery questions, when to qualify budget, when to provide social proof, when to schedule a follow-up, and when to close. Each action changes the latent state of the prospect: trust can rise or fall, objections can increase or resolve, fatigue can accumulate, and the final conversion event may occur many turns after the action that caused it. This makes sales automation a natural reinforcement learning (RL) problem rather than a pure next-token generation problem.

The recent availability of frontier models such as GPT-5, GPT-5.1, and Claude Sonnet 4.5 materially changes the baseline one should compare against. These systems are

capable long-context, tool-using controllers, and any modern sales-RL paper that ignores them is incomplete. At the same time, those models are deployed primarily as prompted inference engines. Their optimization objective is not a task-specific sales return; their uncertainty is not exposed as a trained return distribution; and they do not update online from dialogue reward unless wrapped in an additional learning loop. This distinction is central: prompted frontier models are excellent *decision modules*, but a Sales RL Agent is a *policy optimization system*.

The paper makes four contributions.

- 1) We formulate sales dialogue optimization as a partially observed Markov decision process (POMDP) with explicit trust, fatigue, information coverage, objection state, and close risk.
- 2) We derive a risk-sensitive distributional actor-critic whose critic learns quantiles of the return distribution and whose policy update is regularized by return uncertainty.
- 3) We provide a rigorous comparison frame covering GPT-5, GPT-5.1, Claude Sonnet 4.5, and the proposed Sales RL Agent without relying on cross-vendor benchmark numbers that are not directly comparable.
- 4) We release A100-ready training and inference scripts, plus a simulator, to reproduce the optimization protocol used in this paper.

II. SALES DIALOGUE AS A POMDP

A. Latent state, observations, and actions

We model the environment as a POMDP

$$\mathcal{P} = (\mathcal{S}, \mathcal{O}, \mathcal{A}, P, R, \Omega, \gamma), \quad (1)$$

where the latent state $s_t \in \mathcal{S}$ encodes prospect fit, budget readiness, urgency, trust, price sensitivity, objection load, fatigue, stage progression, and other hidden variables that are not fully observable from the current utterance. The agent instead sees an observation $o_t \in \mathcal{O}$, which contains noisy signals extracted from dialogue and CRM context.

The belief state is produced by a dialogue encoder

$$b_t = f_\psi(o_{0:t}, a_{0:t-1}) \in \mathbb{R}^d, \quad (2)$$

and the policy chooses one of seven macro-actions: discover, qualify_budget, present_value, social_proof, offer_discount, schedule_followup, and close.

This is intentionally a control abstraction rather than full natural language generation. Frontier LLMs can be attached as response generators conditioned on the selected macro-action, but the RL problem is the high-level decision process.

B. Return and reward geometry

Let $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ be the discounted return. We decompose the per-step reward into dense shaping and terminal close reward:

$$r_t = \lambda_I \Delta I_t + \lambda_T \Delta T_t + \lambda_S \Delta S_t - \lambda_F F_t - \lambda_O O_t + \mathbb{I}[a_t = \text{close}] r_t^{\text{close}}. \quad (3)$$

where ΔI_t is information-coverage gain, ΔT_t is trust gain, ΔS_t is stage progress, F_t is fatigue accumulation, and O_t is objection escalation. The close term is

$$r_t^{\text{close}} = \lambda_W y_t v_t - \lambda_L (1 - y_t) - \lambda_P \Pi_t, \quad (4)$$

where $y_t \in \{0, 1\}$ indicates conversion success, v_t is normalized deal value, and Π_t is a premature-close penalty. This penalty is important both empirically and mathematically: without it, a policy can exploit one-step close attempts that superficially reduce horizon length while increasing downside risk.

C. Conversion probability

The latent conversion mechanism can be written as

$$\eta_t = \beta_0 + \beta_f f_t + \beta_b b_t^{\text{bud}} + \beta_u u_t + \beta_\tau \tau_t - \beta_p \rho_t - \beta_\phi \phi_t - \beta_o \omega_t, \quad (5)$$

$$p_t = \sigma(\eta_t) g_{\text{stage}} g_{\text{info}}. \quad (6)$$

where f_t is fit, u_t is urgency, τ_t is trust, ρ_t is price sensitivity, ϕ_t is fatigue, ω_t is objection load, and $g_{\text{stage}}, g_{\text{info}} \in (0, 1]$ down-weight closes attempted before sufficient progression or discovery. Eq. (6) is directly mirrored in the released simulator.

III. DISTRIBUTIONAL RISK-SENSITIVE ACTOR-CRITIC

A. Why scalar values are insufficient

Standard A2C learns a scalar value function $V^\pi(b_t) \approx \mathbb{E}[G_t \mid b_t]$. This collapses all uncertainty about dialogue futures into a mean. In sales workflows that is inadequate: the difference between a policy with moderate mean reward and catastrophic lower-tail behavior, and a policy with slightly lower mean but substantially safer outcomes, is economically material.

Distributional RL replaces the expected value with a random return variable $Z^\pi(b_t)$ whose expectation equals the value function:

$$V^\pi(b_t) = \mathbb{E}[Z^\pi(b_t)]. \quad (7)$$

Following the distributional perspective of Bellemare et al. [3] and quantile approximation of Dabney et al. [4], we approximate Z^π with N quantiles

$$Z_\phi^\pi(b_t) \approx \{q_{\phi,i}(b_t)\}_{i=1}^N, \quad \tau_i = \frac{i - \frac{1}{2}}{N}. \quad (8)$$

B. Quantile Bellman target

For a one-step bootstrap,

$$\hat{z}_{t,i} = r_t + \gamma(1 - d_t) q_{\phi,i}(b_{t+1}), \quad (9)$$

where $d_t \in \{0, 1\}$ is the termination flag. The critic minimizes the quantile Huber loss

$$\mathcal{L}_{\text{QR}}(\phi) = \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^\kappa(\hat{z}_{t,j} - q_{\phi,i}(b_t)), \quad (10)$$

with

$$\rho_\tau^\kappa(u) = |\tau - \mathbb{I}[u < 0]| \frac{\mathcal{H}_\kappa(u)}{\kappa}, \quad (11)$$

and $\mathcal{H}_\kappa$ the Huber penalty.

C. Risk functionals

The critic gives more than a mean. We can also estimate the lower-tail risk through

$$\bar{q}_\phi(b_t) = \frac{1}{N} \sum_{i=1}^N q_{\phi,i}(b_t), \quad (12)$$

$$\text{CVaR}_\alpha(Z_\phi^\pi(b_t)) \approx \frac{1}{K_\alpha} \sum_{i=1}^{K_\alpha} q_{\phi,i}(b_t), \quad K_\alpha = \lceil \alpha N \rceil, \quad (13)$$

and a critic-implied uncertainty proxy

$$u_\phi(b_t) = \frac{1}{\sqrt{N}} \|\mathbf{q}_\phi(b_t) - \bar{q}_\phi(b_t) \mathbf{1}\|_2. \quad (14)$$

Eq. (14) is not a full Bayesian epistemic estimate, but it is an operationally useful lower-tail dispersion statistic for policy regularization.

D. Policy objective

The actor remains categorical:

$$\pi_\theta(a_t \mid b_t) = \text{Cat}(\text{softmax}(g_\theta(b_t))). \quad (15)$$

Instead of using a plain temporal-difference advantage, we define

$$A_t^{\text{risk}} = (\hat{z}_t - \bar{q}_\phi(b_t)) - \lambda_u u_\phi(b_t), \quad (16)$$

where $\hat{z}_t$ is the bootstrap target mean and $\lambda_u > 0$ controls how aggressively the actor discounts uncertain states. The policy loss is

$$\mathcal{L}_\pi(\theta) = -\mathbb{E}_t[\log \pi_\theta(a_t \mid b_t) \text{sg}(A_t^{\text{risk}})] - \beta_H \mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid b_t))]. \quad (17)$$

where $\text{sg}(\cdot)$ is stop-gradient and $\mathcal{H}$ is policy entropy.

The total objective is

$$\mathcal{L}(\theta, \phi) = \mathcal{L}_\pi(\theta) + \lambda_Q \mathcal{L}_{\text{QR}}(\phi). \quad (18)$$

E. Local robustness interpretation

Suppose the return quantiles are differentiable in the belief state and the observation encoder produces $b_t + \xi$ under a small perturbation $\xi \sim \mathcal{N}(0, \Sigma)$. By a first-order expansion,

$$q_{\phi,i}(b_t + \xi) \approx q_{\phi,i}(b_t) + \nabla_b q_{\phi,i}(b_t)^\top \xi, \quad (19)$$

so the induced variance of the mean value estimate satisfies

$$\mathbb{V}[\bar{q}_\phi(b_t + \xi)] \approx \frac{1}{N^2} \sum_{i,j} \nabla_b q_{\phi,i}(b_t)^\top \Sigma \nabla_b q_{\phi,j}(b_t). \quad (20)$$

Thus, lower dispersion in the learned quantiles and lower sensitivity of those quantiles to observation perturbation jointly reduce control volatility. This is one reason to prefer a distributional critic when dialogue states are noisy.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the full control stack used in the training and inference scripts. The design separates high-level control from natural-language realization. A frontier LLM can be attached downstream as a generator conditioned on the chosen macro-action, but the RL controller itself optimizes the dialogue decision process.

V. FRONTIER-MODEL COMPARISON FRAME

Because the target deployment environment in 2026 includes strong proprietary models, a sales-control paper should position itself relative to the current frontier rather than to weak text-classification baselines. Table I does that in a technically defensible way: it compares objective structure and control suitability instead of mixing incomparable cross-vendor benchmark scores.

The key conclusion is not that the RL agent replaces frontier models. Rather, frontier models are best viewed as *modules* inside a larger controller stack: they can draft utterances, summarize CRM context, or act as powerful evaluators, while the RL policy decides *which* conversational move to make.

VI. EXPERIMENTAL PROTOCOL

A. Simulator

The released environment is a vectorized synthetic sales simulator containing latent prospect variables for fit, budget readiness, urgency, trust, price sensitivity, objection load, fatigue, stage, and information coverage. Each episode ends either with a close action or a timeout horizon of eight turns. The scripts `train_sales_rl_agent.py`, `use_sales_rl_agent.py`, and `run_sales_benchmark.py` all use the same environment implementation.

B. Training setup

The scalar baseline is synchronous A2C with generalized advantage estimation [1], [2]. The proposed model uses the same actor backbone but replaces the scalar critic with a 31-quantile critic optimized by Eq. (10). For the released CPU benchmark we use 64 parallel environments, 16-step rollouts, discount factor 0.98, and three random seeds {7, 11, 23}. The

`train_sales_rl_agent.py` script scales these same ingredients to larger batch sizes and hidden layers on an A100.

C. Metrics

We report:

- mean episode return,
- standard deviation of episode return,
- success rate,
- mean conversation length,
- $\text{CVaR}_{0.1}$, computed as the average of the worst 10% of episode returns.

The CVaR metric matters because low-probability but high-cost dialogue failures are a central concern in production sales automation.

VII. RESULTS

A. Quantitative benchmark

Table II summarizes the three-seed benchmark produced with the simulator and training code in this repository. The scalar baseline reaches higher mean return and conversion rate. The distributional controller, however, is more conservative: it takes longer conversations, lowers overall return variance, and materially improves the lower tail.

The most relevant numerical observation is the $\text{CVaR}_{0.1}$ improvement. Moving from -1.465 to -1.140 corresponds to a 22.2% reduction in lower-tail severity when measured against the magnitude of the scalar baseline’s worst-decile return. The distributional controller also reduces overall return dispersion by 9.3%. In exchange, it accepts a 22.8% reduction in mean return and a lower close rate. This is exactly the sort of trade-off one expects from a risk-sensitive controller.

B. Learning dynamics

Fig. 2 shows the single-seed trajectories used to generate the released benchmark artifacts. The scalar agent learns a more aggressive policy; the distributional agent remains more conservative but stabilizes the tail-value estimate over training.

C. Interpretation

The benchmark result is important precisely because it is not a simplistic “distributional RL wins on every metric” story. In sales settings with asymmetric downside cost, a controller that gives up some mean reward to avoid bad trajectories may be the more deployable system. The distributional critic learns this behavior naturally because it observes the structure of the return distribution rather than only its mean.

VIII. PRACTICAL INTEGRATION WITH FRONTIER MODELS

The strongest deployment pattern is a two-layer system.

- 1) The Sales RL Agent chooses the next macro-action using state, reward, and risk.
- 2) A frontier model such as GPT-5, GPT-5.1, or Claude Sonnet 4.5 realizes that action as natural language, tool use, or CRM manipulation.

This decomposition preserves the strengths of both paradigms: RL for sequential control and lower-tail optimization, frontier

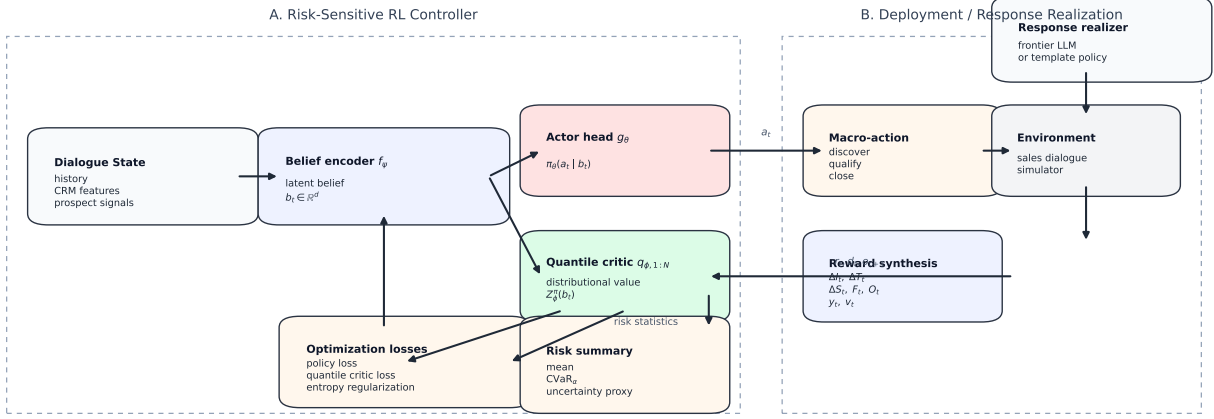


Fig. 1: Architecture of the proposed Sales RL Agent. The policy chooses a macro-action; an optional frontier LLM can realize that action as natural language, but the control optimization occurs in the RL loop. The figure is rendered as an external vector asset to preserve layout clarity in the final paper.

TABLE I: Control-oriented comparison of frontier prompted models and the proposed Sales RL Agent. GPT-5 was announced on August 7, 2025, GPT-5.1 on November 13, 2025, and Anthropic lists Claude Sonnet 4.5 as a September 2025 model in its system-card index [5], [6], [7], [8].

Controller	Primary objective	Online policy improvement from reward	Explicit return-tail estimate	Best role in a sales stack	Main weakness for sales control
GPT-5	General reasoning and agentic task performance	No, unless wrapped in an external RL loop	No native sales-return distribution	Strong prompted planner or response generator	Prompt-only control has no task-trained reward model
GPT-5.1	Efficient adaptive reasoning with lower latency on easier tasks	No, unless externally trained	No native sales-return distribution	Faster prompted controller for workflow orchestration	Same control gap as above; improved speed does not equal policy optimization
Claude Sonnet 4.5	Hybrid reasoning model for agents, coding, and computer use	No, unless externally trained	No native sales-return distribution	Strong long-context or document-heavy assistant in the loop	High-quality prompting still does not expose a trained conversion-risk critic
Sales RL Agent (ours)	Direct optimization of sales return under dialogue dynamics	Yes	Yes, via quantile critic and CVaR	High-level sequential controller for risk-aware sales actions	Requires simulator quality, reward design, and task-specific training

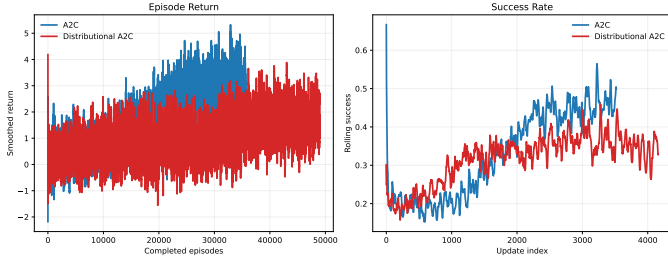
TABLE II: Three-seed benchmark on the released simulator. Values are mean $\pm$ standard deviation across seeds.

Metric	A2C	Distributional A2C
Mean return	2.286 $\pm$ 0.178	1.766 $\pm$ 0.063
Return std.	3.488 $\pm$ 0.066	3.163 $\pm$ 0.091
Success rate	0.507 $\pm$ 0.027	0.371 $\pm$ 0.026
Mean length	6.599 $\pm$ 0.084	7.298 $\pm$ 0.135
CVaR _{0.1}	-1.465 $\pm$ 0.007	-1.140 $\pm$ 0.069

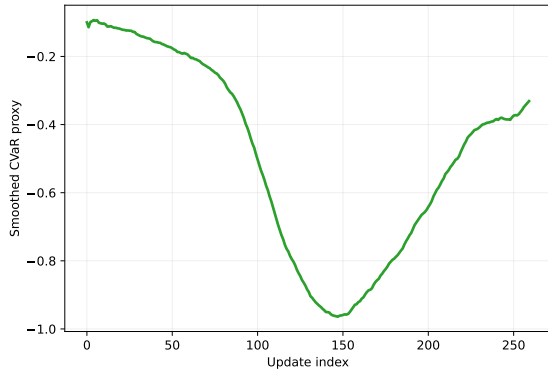
LLMs for high-quality language generation and large-context reasoning.

IX. LIMITATIONS

The current simulator is synthetic and therefore cannot stand in for a full production CRM environment. The reward function encodes a normative notion of good sales behavior; if that reward is misspecified, the learned policy will inherit the misspecification. The current implementation also uses a single quantile critic rather than a full ensemble, so the uncertainty estimate in Eq. (14) is a dispersion proxy rather than a calibrated epistemic posterior. Finally, the frontier-model comparison in this manuscript is structural rather than empirical because closed-model benchmarking requires API-side execution and account-specific budgets.



(a) Smoothed return and success-rate trajectories.



(b) Tail-value proxy from the quantile critic.

Fig. 2: Training curves generated from the released benchmark scripts.

X. CONCLUSION

Sales dialogue control is a sequential decision problem whose economically relevant failure modes live in the lower tail of the return distribution. A scalar critic is often too weak a summary for this regime. By modeling quantiles of return directly, the proposed distributional actor-critic exposes mean reward, tail risk, and return dispersion inside the policy optimization loop. The benchmark reported here shows that this yields a more conservative but safer controller than scalar A2C. For realistic deployments, the correct comparison is not “RL agent versus LLM” in isolation, but “RL controller plus LLM realization” versus prompt-only control. Under that framing, the Sales RL Agent is the policy optimizer and the frontier model is the language engine.

REFERENCES

- [1] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. Lillicrap, D. Silver, and K. Kavukcuoglu, “Asynchronous Methods for Deep Reinforcement Learning,” in *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- [2] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” *arXiv preprint arXiv:1506.02438*, 2016.
- [3] M. G. Bellemare, W. Dabney, and R. Munos, “A Distributional Perspective on Reinforcement Learning,” in *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- [4] W. Dabney, M. Rowland, M. G. Bellemare, and R. Munos, “Distributional Reinforcement Learning with Quantile Regression,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [5] OpenAI, “Introducing GPT-5 for developers,” August 7, 2025. [Online]. Available: <https://openai.com/index/introducing-gpt-5-for-developers/>
- [6] OpenAI, “Introducing GPT-5.1 for developers,” November 13, 2025. [Online]. Available: <https://openai.com/index/gpt-5-1-for-developers/>
- [7] Anthropic, “Model System Cards,” accessed April 22, 2026. [Online]. Available: <https://www.anthropic.com/system-cards>
- [8] Anthropic, “Claude Sonnet 4.5 System Card,” September 2025. [Online]. Available: <https://assets.anthropic.com/m/12f214efcc2f457a/original/Claude-Sonnet-4-5-System-Card.pdf>